Instituto Politécnico Nacional

Escuela Superior de Cómputo

Unidad Académica:

Sistemas Distribuidos

Profesor:

Carreto Arellano Chadwick

Práctica 03: "Escalando a múltiples clientes y un servidor"

Alumna:

Maldonado Bedolla Daniela del Carmen

Grupo:

7CM1

4 de Marzo de 2026

Índice

INTRODUCCIÓN **3**

DESARROLLO **4**

PROBLEMÁTICA 4
PROPUESTA DE SOLUCIÓN 4
IMPLEMENTACIÓN 5
Servidor *5*
Cliente *7*

EJECUCIÓN Y RESULTADOS **8**

EJECUCIÓN DE LA SIMULACIÓN CLIENTE/SERVIDOR 8
COMPARACIÓN DE RESULTADOS: 13

CONCLUSIONES **15**

ANEXO **16**

CÓDIGO DEL SERVIDOR 16
CÓDIGO DE CLIENTE 17

INTRODUCCIÓN

Escalamos este proyecto desde un modelo concurrente local hacia una arquitectura distribuida más completa de múltiples clientes y un servidor. En la práctica 1, la compra de boletos se resolvió en un solo programa con hilos compartiendo memoria, lo que permitió estudiar condiciones de carrera y sincronización en un entorno controlado. En la práctica 2, el sistema pasó a cliente/servidor, donde el servidor centralizó el estado de los asientos y los clientes enviaban solicitudes por socket para apartar y comprar.

En esta nueva etapa, el modelo se amplía para que varios puntos de acceso cliente operen al mismo tiempo contra un único servidor. Cada cliente representa una fuente independiente de compradores concurrentes, y el servidor coordina el inicio global de la venta mediante registro, confirmación de disponibilidad y señal de arranque sincronizada. Con esto, la simulación se acerca más a un escenario real de alta demanda distribuida.

A nivel de control, el servidor mantiene la autoridad total sobre el recurso crítico (asientos), aplica reglas por tipo de comprador y zonas permitidas, y protege la sección crítica con mecanismos de bloqueo para conservar consistencia. Además, se incorporan reservas temporales y validaciones de compra para evitar conflictos, doble venta y estados inválidos durante la ejecución.

Finalmente, la práctica conserva la observabilidad con visualización en tiempo real y métricas de desempeño, lo que permite analizar no solo la concurrencia de hilos, sino también el comportamiento del sistema bajo múltiples clientes conectados en paralelo. Esto facilita evaluar integridad, coordinación y rendimiento en un contexto distribuido más exigente.

Desarrollo

Problemática

En esta etapa, el problema se amplió a un esquema de **múltiples clientes y un servidor** con mayor volumen y reglas más realistas. Ya no se trabaja con 1,000 asientos, sino con una capacidad mayor de **1,500 asientos (30 x 50)**, distribuidos por secciones. Además, se incorporaron **tipos de compradores** (platino, preferente y normal), cada uno con permisos distintos de acceso a zonas, lo que agrega una capa de prioridad funcional durante la competencia.

La dificultad principal sigue siendo la concurrencia, pero ahora en un contexto más exigente por red y con varios puntos de acceso al mismo tiempo. Los riesgos críticos son:

- Múltiples clientes pueden intentar reservar o comprar en paralelo sobre el mismo grupo de asientos.
- Puede haber diferencia temporal entre el estado local de cada cliente y el estado real del servidor.
- Si una reserva no se confirma, el asiento puede quedar en estado intermedio si no se libera correctamente.
- Con tipos de comprador distintos, se requiere aplicar reglas de prioridad sin romper consistencia.
- Si hay menos compradores que asientos disponibles, la simulación puede quedar esperando indefinidamente si no existe un cierre coordinado.

Por eso, el reto ya no es solo ejecutar hilos concurrentes, sino coordinar clientes distribuidos, respetar prioridades por tipo y mantener un estado único, consistente y cerrable en el servidor.

Propuesta de solución

La solución se basa en un modelo cliente-servidor concurrente donde el servidor centraliza todo el control:

- El servidor acepta múltiples conexiones TCP y atiende cada cliente en hilos independientes.

- Cada cliente se registra, indica tipo y cantidad de compradores, y espera una señal global de inicio.
- El servidor sincroniza el arranque cuando se alcanza el número de clientes esperados (REGISTER + READY + START).
- Los compradores se ejecutan como hilos en cada cliente y envían solicitudes por socket (REQUEST_TICKET, PURCHASE).
- Los asientos usan estados FREE, RESERVED, SOLD, con bloqueos por zona para evitar doble venta.
- Se manejan reservas con TTL para liberar asientos no confirmados y evitar bloqueos permanentes.
- Se aplican reglas por tipo:
    - platino: puede comprar en cualquier sección,
    - preferente: preferente o normal,
    - normal: solo normal.
- Se conserva visualización en servidor (estado de asientos por color y secciones) y métricas de tiempo para análisis.

Con este enfoque, se mantiene la integridad del sistema bajo alta concurrencia distribuida, se respeta la política de prioridad por tipo y se obtiene una ejecución ordenada tanto del lado cliente como del servidor.

Implementación

Servidor

El servidor mantiene el control total del estado de venta y coordina a múltiples clientes. Estos fueron los 3 ajustes críticos:

1. **Arranque sincronizado para múltiples clientes (REGISTER/READY/START)**
   El servidor ya no inicia cuando quiere un solo cliente; ahora espera un número de clientes esperado, registra su conexión y arranca cuando todos están listos.

```
if (not self.start_event.is_set()
        and connected_count >= self.expected_clients
        and ready_count >= self.expected_clients):
    self.start_event.set()
    self.ticket_state.open_sales()
```

- expected_clients: define cuántos clientes deben conectarse.
- register_client(...): guarda quién se conectó.
- mark_ready(...): confirma que cada cliente está listo.
- start_event: libera el inicio simultáneo.

2. **Zonas y prioridad por tipo de comprador**

   Se dividieron asientos por secciones y el servidor aplica reglas de acceso según tipo.

```
ALLOWED_ZONES_BY_TYPE = {
    "platino": ["PLATINO", "PREFERENTE", "NORMAL
    "preferente": ["PREFERENTE", "NORMAL"],
    "normal": ["NORMAL"],
}
```

- Filas 1-3: PLATINO
- Filas 4-7: PREFERENTE
- Filas 8-30: NORMAL
- Reglas:
  - platino: cualquier zona
  - preferente: preferente + normal
  - normal: solo normal

3. **Sección crítica protegida por zona (evitar doble venta)**
   Para evitar conflictos concurrentes, se usan locks por zona en reserva y compra.

```
zone_lock = self.zone_locks[zone]
acquired = zone_lock.acquire(timeout=0.02)
...
self.seat_status[row][col] = "RESERVED"
...
zone_lock.release()
```

- zone_locks: lock independiente por sección.
- Reserva: FREE -> RESERVED
- Compra: RESERVED -> SOLD
- Si no hay confirmación, TTL libera asiento.

Cliente

El cliente representa un punto de acceso y genera compradores concurrentes. Estos fueron los 3 ajustes críticos:

1. **Cliente parametrizable por tipo y cantidad**
   Ahora el cliente se ejecuta simple, con dos argumentos principales: tipo y cantidad.

```
parser.add_argument("client_type")
parser.add_argument("buyers", type=int)
normalized_type = normalize_client_type(args.client_type)
```

- Ejemplo: python cliente.py A 100
- A/B/C se mapean a platino/normal/preferente.

2. **Handshake de coordinación con servidor**

   Antes de crear compradores, el cliente se registra y espera arranque global.

```
send_control_message(sock_file, {"type": "REGISTER", ...})
send_control_message(sock_file, {"type": "READY", ...})
start_response = read_control_message(sock_file)  # espera START
```

- Envía REGISTER
- Envía READY
- Espera START

3. **Generación concurrente de compradores por punto de acceso**

   Cada cliente crea sus propios hilos compradores y cada hilo compite por asiento vía socket.

```
for buyer_number in range(1, args.buyers + 1):
    thread = threading.Thread(
        target=buyer_worker,
        args=(buyer_number, args.host, args.port, client_id, normalized_type),
    )
    thread.start()
```

- buyer_worker(...): ciclo de intento de reserva y compra.
- send_request(...): comunicación JSON con servidor.
- Se miden tiempos y resultados por cliente

Ejecución y Resultados

Ejecución de la simulación cliente/servidor

En esta práctica, la ejecución inicia con el servidor, que configura el foro en **30x50**, carga las secciones de asientos (**platino, preferente y normal**) y queda a la espera del número de clientes definido al arranque.
En la terminal se observa el puerto de escucha, el total de asientos y la cantidad de clientes esperados antes de habilitar la venta.

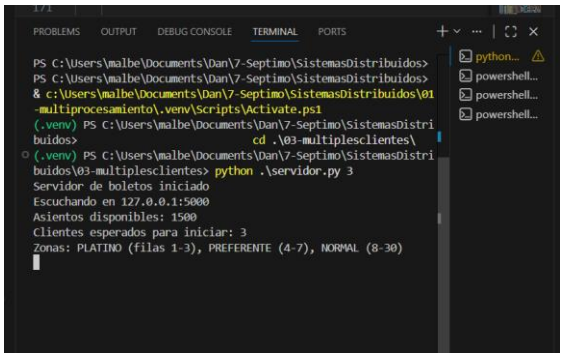


Figura 1. Arranque del servidor en terminal

Con el servidor en espera, la interfaz gráfica muestra el mapa de asientos dividido en tres secciones visibles, con separación entre bloques y etiquetas por zona. En esta etapa la venta aún no inicia, por lo que todos los asientos se mantienen en estado libre.

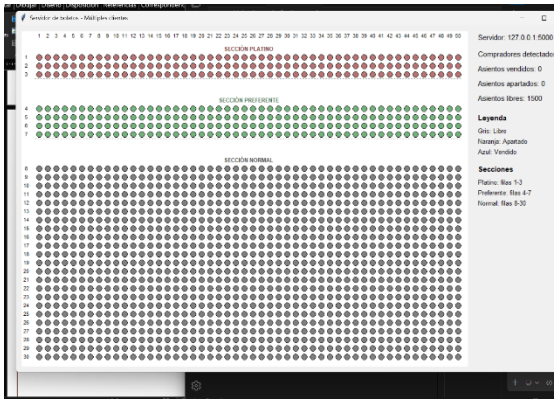


Figura 2. Visualización inicial del servidor en espera

A continuación, se ejecutan tres clientes desde terminal, cada uno representando un punto de acceso con tipo de comprador distinto (por ejemplo: platino, preferente y

normal). Cada cliente se registra en el servidor, envía señal de listo y espera la orden de inicio global (START).

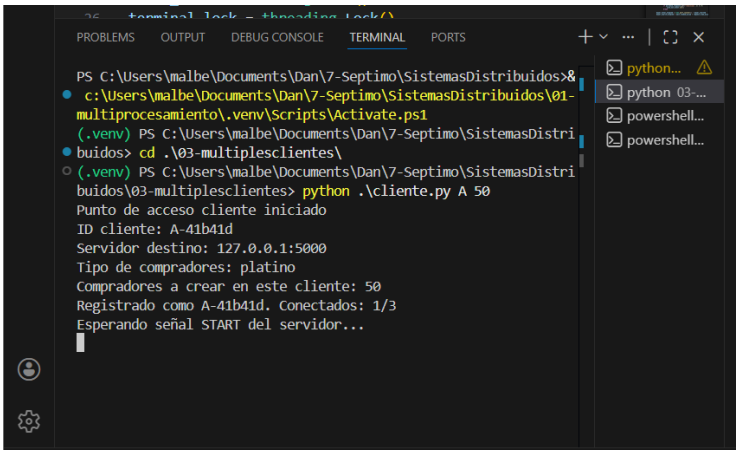


Figura 3. Ejecución del cliente platino

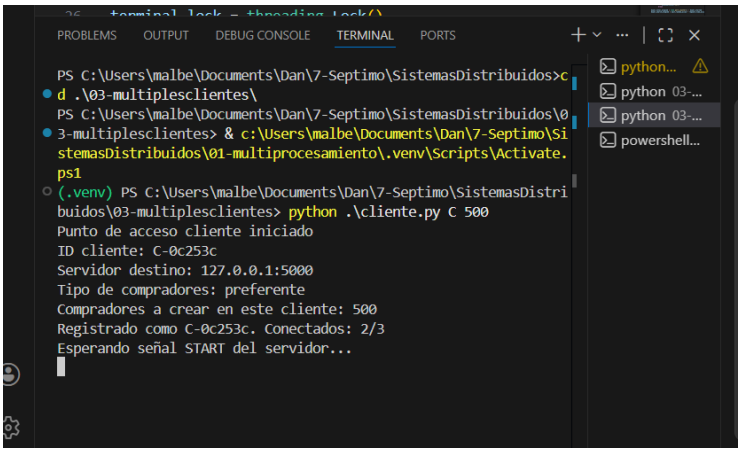


Figura 4. Ejecución del cliente preferente

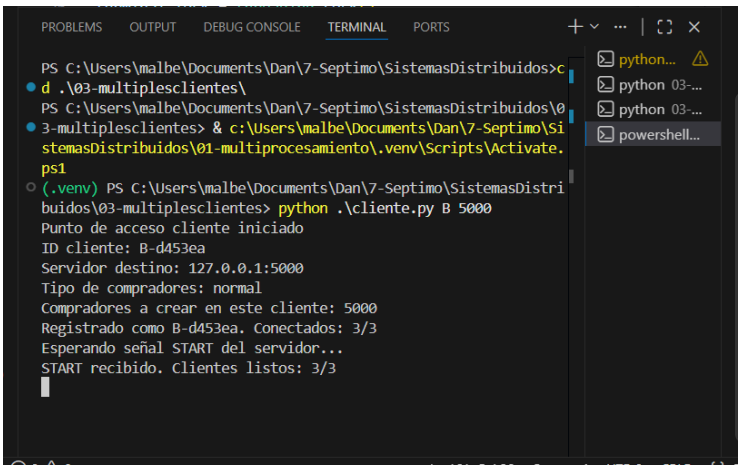


Figura 5. Ejecución del cliente normal

Cuando todos los clientes esperados están conectados y listos, el servidor habilita la venta y comienza la concurrencia real. Durante la simulación, el estado de cada asiento cambia de forma continua:

- libre (color base por sección),
- apartado (naranja),
- comprado (azul).

Esto permite observar visualmente el flujo FREE -> RESERVED -> SOLD en tiempo real.

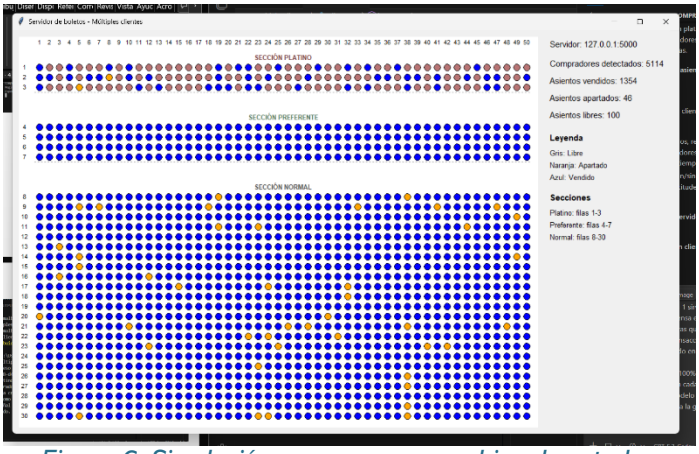


Figura 6. Simulación en curso con cambios de estado

En este escenario se usó una cantidad baja de compradores platino para comprobar el comportamiento por prioridad de zonas: al finalizar, algunos asientos de la sección platino pueden quedar vacíos, validando que compradores de menor prioridad no ocupan zonas no permitidas.

Figura 7. Resultado final con asientos platino sin vender

Finalmente, el servidor y cada cliente imprimen su resumen de ejecución.

- **Servidor:** asientos vendidos, reservas activas/liberadas, compradores detectados, solicitudes procesadas y tiempos promedio.
- **Clientes:** compradores con/sin compra, tiempos locales y métricas de solicitudes (request_ticket y purchase).

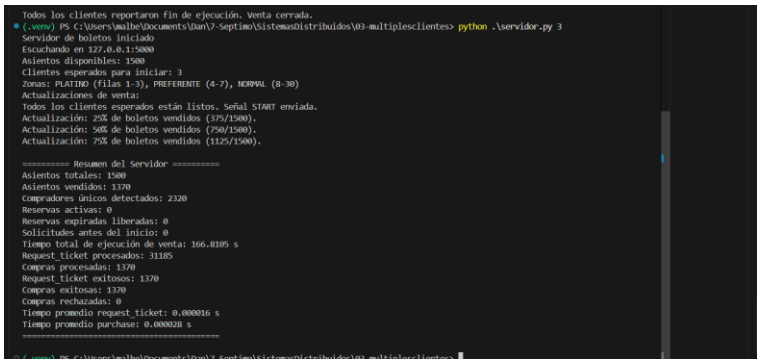


Figura 8. Resumen final del servidor

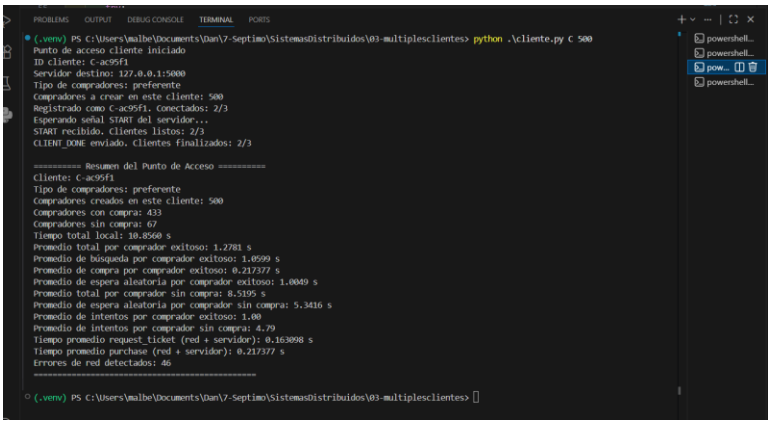


Figura 9. Resumen final de un cliente con compradores Platino (A)

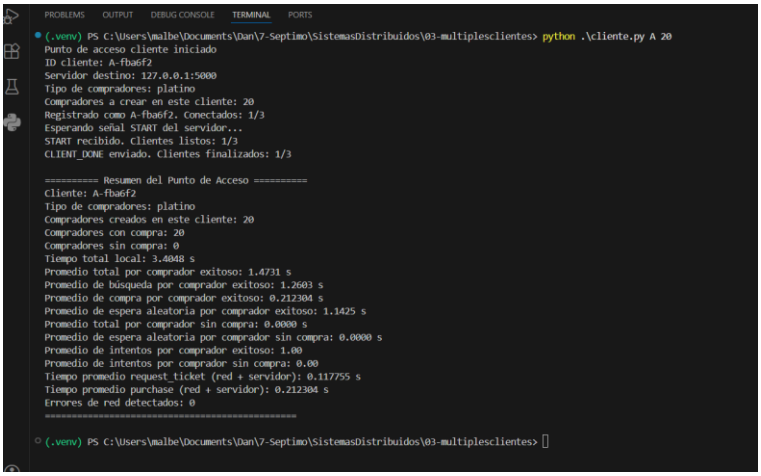
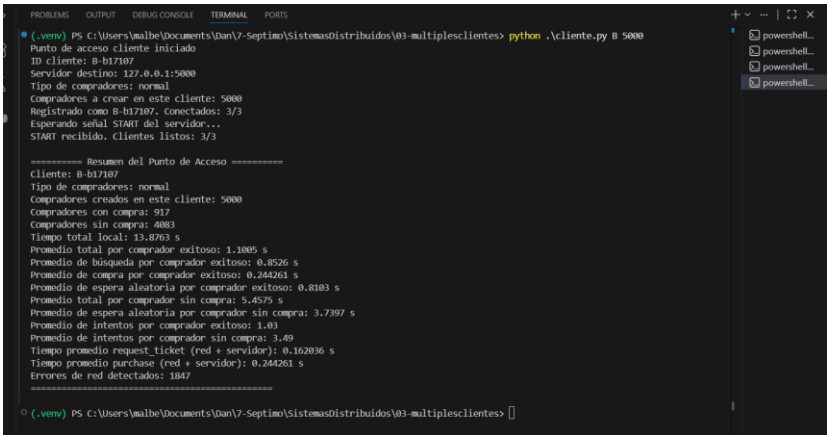


Figura 10. Resumen final de un cliente con compradores Preferente (C)



```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

(.venv) PS C:\Users\malbe\Documents\Dan\7-Septimo\SistemasDistribuidos\03-multiplesclientes> python .\cliente.py B 5000
Punto de acceso cliente iniciado
ID cliente: B-b17107
Servidor destino: 127.0.0.1:5000
Tipo de compradores: normal
Compradores a crear en este cliente: 5000
Registrado como B-b17107. Conectados: 3/3
Esperando señal START del servidor...
START recibido. Clientes listos: 3/3

========= Resumen del Punto de Acceso =========
Cliente: B-b17107
Tipo de compradores: normal
Compradores creados en este cliente: 5000
Compradores con compra: 917
Compradores sin compra: 4083
Tiempo total local: 13.8763 s
Promedio total por comprador exitoso: 1.1005 s
Promedio de búsqueda por comprador exitoso: 0.8526 s
Promedio de compra por comprador exitoso: 0.244261 s
Promedio de espera aleatoria por comprador exitoso: 0.8103 s
Promedio total por comprador sin compra: 5.4575 s
Promedio de espera aleatoria por comprador sin compra: 3.7397 s
Promedio de intentos por comprador exitoso: 1.03
Promedio de intentos por comprador sin compra: 3.49
Tiempo promedio request_ticket (red + servidor): 0.162036 s
Tiempo promedio purchase (red + servidor): 0.244261 s
Errores de red detectados: 1847
==============================================

(.venv) PS C:\Users\malbe\Documents\Dan\7-Septimo\SistemasDistribuidos\03-multiplesclientes>
```

Figura 11. Resumen final de un cliente con compradores Normales (B)

Comparación de resultados:

En ambas prácticas se cumplió el objetivo principal: vender la totalidad de los 1,000 boletos sin incurrir en sobreventa y con un cierre correcto del sistema. Asimismo, los hitos de avance (25 %, 50 %, 75 % y 100 %) se alcanzaron en orden secuencial en los dos casos, lo que confirma la consistencia del conteo global de asientos vendidos independientemente de la arquitectura empleada.

Los tiempos totales de las tres prácticas (13.35 s en P1, 2.97 s en P2 y 13.88 s en P3) no deben interpretarse como medidas directas de rendimiento equivalente, ya que las metodologías de medición y las condiciones de carga difieren sustancialmente. En la práctica 1 se contabiliza el ciclo completo de todos los hilos compitiendo en memoria compartida hasta el último asiento; en la práctica 2 el tiempo corresponde al intervalo de procesamiento del servidor desde la primera hasta la última venta confirmada con un solo cliente conectado; y en la práctica 3 el servidor atiende simultáneamente tres puntos de acceso con un total de 5,520 compradores (hilos) distribuidos en tipos platino (20), preferente (500) y normal (5,000), compitiendo por 1,500 asientos divididos en zonas con restricciones de prioridad.

La práctica 3 nos muestra alta afluencia: la demanda (5,520 compradores) supera con creces la oferta (1,500 asientos), lo que genera una presión de contención significativa. Los compradores platino, al ser pocos y tener acceso exclusivo a las primeras filas, logran el 100 % de éxito sin errores de red ni reintentos. Los compradores preferente presentan una tasa moderada de éxito (433 de 500, 86.6 %) con solo 46 errores de red, beneficiándose de su acceso a dos zonas (preferente y normal). En contraste, los compradores normales —con 5,000 hilos compitiendo únicamente por los asientos de la zona general— logran apenas 917 compras (18.3 %) y acumulan 1,847 errores de red, reflejo directo de la saturación masiva de conexiones simultáneas al servidor. El promedio de intentos por comprador sin compra (3.49) y su tiempo promedio elevado (5.46 s) confirman que los hilos normales iteran repetidamente hasta que la venta se cierra por agotamiento de asientos elegibles.

Cliente	Tipo	Compradores	Con compra	Sin compra	Tiempo local	Errores de red
A	platino	20	20	0	3.40 s	0
C	preferente	500	433	67	10.86 s	46
B	normal	5,000	917	4,083	13.88 s	1,847

En síntesis, la práctica 1 permite observar la competencia intensa entre hilos en memoria compartida y el impacto de la contención sobre los tiempos individuales. La práctica 2 demuestra control transaccional centralizado con cierre ordenado en un entorno distribuido de un solo cliente. La práctica 3 lleva este modelo al extremo, exponiendo el servidor a múltiples clientes con cargas asimétricas y prioridades diferenciadas: los resultados validan que la arquitectura mantiene la integridad de las transacciones y respeta estrictamente la jerarquía de zonas incluso bajo presión masiva,

aunque el costo se manifiesta en errores de red y tiempos prolongados para los compradores de menor prioridad que saturan las conexiones disponibles.

Conclusiones

Después de completar las tres prácticas, entiendo con claridad la progresión entre concurrencia local, cliente-servidor con un solo punto de acceso y múltiples clientes simultáneos. En la práctica 1 aprendí a controlar condiciones de carrera en memoria compartida; en la práctica 2 confirmé que la clave de un sistema distribuido es definir una autoridad centralizada del estado; y en la práctica 3 comprobé que esa autoridad puede sostener la integridad incluso al multiplicar puntos de acceso, diferenciar prioridades y aumentar la escala.

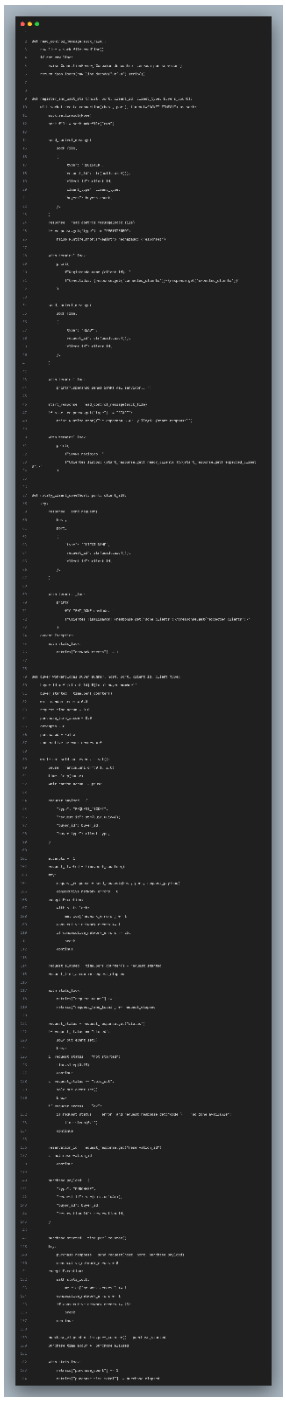
El paso a múltiples clientes exigió un protocolo de sincronización más robusto (registro, inicio coordinado, finalización por cliente), zonas exclusivas por tipo, cierre automático por demanda elegible agotada y manejo de desconexiones sin comprometer la estabilidad. A pesar de incrementar los asientos un 50 % (de 1,000 a 1,500) y la demanda a más de 5,500 compradores concurrentes, el servidor mantuvo cero sobreventa y cierre limpio. El tiempo total de la práctica 3 (13.88 s) es comparable al de la práctica 1 (13.35 s), lo que confirma que el cuello de botella no está en la lógica transaccional sino en la saturación de conexiones: los 20 compradores platino terminaron en 3.4 s sin errores, mientras que los 5,000 normales acumularon casi 1,850 errores de red.

La observabilidad —visualización de estados, hitos de venta y métricas diferenciadas por tipo— fue esencial para validar consistencia bajo carga asimétrica. Estas prácticas me permitieron pasar de una visión local de concurrencia a una distribuida: no basta con ejecutar tareas en paralelo, hay que coordinar estados, definir jerarquías de acceso y mantener integridad de datos sin importar cuántos puntos de acceso compitan por los mismos recursos.

Anexo

Nota: Debido a la extensión de ambos códigos, tanto cliente.py como servidor.py se anexarán a la carpeta de entrega, en este reporte solo se incluyeron capturas de secciones iniciales del código.

Código del servidor

Código de cliente

```python
import argparse
import json
import random
import socket
import threading
import time
import uuid

SOCKET_TIMEOUT = 4.0

TIPO_PLATINO = "platino"
TIPO_PREFERENTE = "preferente"
TIPO_NORMAL = "normal"

TYPE_ALIAS = {
    "a": TIPO_PLATINO,
    "b": TIPO_NORMAL,
    "c": TIPO_PREFERENTE,
    TIPO_PLATINO: TIPO_PLATINO,
    TIPO_PREFERENTE: TIPO_PREFERENTE,
    TIPO_NORMAL: TIPO_NORMAL,
}


stats_lock = threading.Lock()
terminal_lock = threading.Lock()
sold_out_event = threading.Event()
threads = []

metrics = {
    "buyers_success": 0,
    "buyers_fail": 0,
    "buyer_total_success": 0.0,
    "buyer_search_success": 0.0,
    "buyer_purchase_success": 0.0,
    "buyer_wait_success": 0.0,
    "buyer_total_fail": 0.0,
    "buyer_wait_fail": 0.0,
    "request_time_total": 0.0,
    "purchase_time_total": 0.0,
    "request_count": 0,
    "purchase_count": 0,
    "attempts_success": 0,
    "attempts_fail": 0,
    "network_errors": 0,
}

sales_start_ts = None
sales_end_ts = None


def monitor_server_health(host, port):
    # Monitorea cierre/agotamiento para cortar todos los hilos clientes de forma limpia.
    while not sold_out_event.is_set():
        try:
            response = send_request(
                host,
                port,
                {
                    "type": "HEALTH",
                    "request_id": str(uuid.uuid4()),
                },
            )

            if response.get("type") == "HEALTH_RESPONSE":
                if response.get("sales_closed"):
                    sold_out_event.set()
                    return

                total_seats = response.get("total_seats")
                sold_count = response.get("sold_count", 0)
                if isinstance(total_seats, int) and total_seats > 0 and sold_count >= total_seats:
                    sold_out_event.set()
                    return
        except Exception:
            # Si falla la consulta puntual, se reintenta en el siguiente ciclo.
            pass

        time.sleep(0.35)


def avg(total, count):
    if count == 0:
        return 0.0
    return total / count


def send_request(host, port, payload):
    data = (json.dumps(payload) + "\n").encode("utf-8")
    with socket.create_connection((host, port), timeout=SOCKET_TIMEOUT) as sock:
        sock.settimeout(SOCKET_TIMEOUT)
        sock.sendall(data)
        response_bytes = b""
        while not response_bytes.endswith(b"\n"):
            chunk = sock.recv(4096)
            if not chunk:
                break
            response_bytes += chunk

    if not response_bytes:
        raise ConnectionError("Respuesta vacía del servidor")

    return json.loads(response_bytes.decode("utf-8").strip())


def send_control_message(sock_file, payload):
    sock_file.write(((json.dumps(payload) + "\n").encode("utf-8"))
    sock_file.flush()

```